

APPLIED MATH 50 LAB 4: INFORMATION THEORY

1. INTRODUCTION

In today's computer lab, we will:

- (1) Learn to count the frequency distribution of words in a passage of text.
- (2) Compare the word frequency distributions in a few (famous and not-so-famous) texts, and compare the measurements to Zipf's law.
- (3) Use this the frequency information to measure *Shannon's entropy* for these texts.

2. SHANNON'S ENTROPY AND THE MAIN POINT OF LAB

Our paper this unit is Shannon's Mathematical Theory of Information, in which Shannon discovers how to measure the information content of, for example, an English text. The heart of Shannon's idea is that we should think of a text as emanating from an information source, which *probabilistically* emits letters or words at predetermined frequencies.

Namely, when you write a term paper for a class, Shannon envisions you as having some set of words at your disposal, and your brain chooses words from this distribution and puts them on the page. To the extent that you are choosing your words in a not-random fashion, the particular order of the words in your brain is the information content of the document.

Shannon measures the information through the *entropy* of the text. Suppose that your brain contains M different words, and that the i th word has a frequency f_i . Then the entropy is given by

$$(1) \quad H = - \sum_{i=1}^M f_i \log_2(f_i).$$

Note that the logarithm here is base 2.¹

As Shannon argues, if you then consider writing a document of N words, the number of possible combinations of words you can choose is

$$\text{\#of Words} = 2^{HN}.$$

Thus the particular combination of words that you have chosen for your essay is one out of 2^{HN} possibilities. Note that the higher the entropy, H , the more possible documents your

¹This, and today's entire lab, is a simplification of the real calculation. In reality, the statistical structure of a language does not just depend on the word frequencies but on the relationship of different words to each other. So for example, a sentence beginning with the word "So" is more likely to have a second word "for" than "example." We are neglecting this effect in today's lab. If you are interested you could turn a more precise analysis into a final project!

brain can put together — and the more unique each particular one is! Shannon defines H with respect to the base 2 logarithm because this way you can imagine coding all of the words in a binary counter, that is, with a series of zeroes and ones, like a computer. *Then the entropy H measures the number of binary digits required to code a word of the language.*

So the question is: What is H ? What is your personal H ? What is the H for the English language? Are they all the same? Or do better (or worse) writers have a higher value for H ? The goal of today's lab is to make some simple measurements to begin to answer these questions.

3. COUNTING WORD FREQUENCY DISTRIBUTIONS

Our goal is to give you the needed tools to compute the entropy of a text. Once you can do this you will measure the entropy of a few representative texts that we give you. In your homework you will measure the entropy of your own writing.

To count the word frequencies, you should use the Matlab program, `wordcount.m`, posted on the course website. The program was written by a contributor to Matlab's public code repository. It takes a filename as input and then outputs the word frequency distribution. We have modified the program a little so that it allows you to plot the outputted data.

The program contains several concepts that are beyond the level of this class and it is not necessary that you understand it in detail. However, we will illustrate how a few of the commands work so that you get the idea. Please work read the points below and try the commands in a script or in the command window.

- (1) First it opens the file. The file **must** be a `.txt` file – it cannot read in Microsoft Word or other such files. You can easily make a `.txt` file by copying the bit of text, and then saving it as a `.txt` file within Microsoft Word. Better yet, use Notepad (Windows) or TextEdit (Mac). On the course website we have given you a `.txt` file of the Gettysburg Address, called `gettysburg.txt`. Read this into Matlab by typing `fid = fopen('gettysburg.txt');` This command opens the file and assigns it to the variable `fid` (short for file identifier).
- (2) Now it converts the file into what is called a *cell array*. This is a data structure that allows Matlab to easily count and manipulate the text in the file. To see what this is, type: `words = textscan(fid, '%s');` The variable `words` is now a cell array, containing the words of the text. If you type `words{1}` then Matlab will spit out all of the individual words of the text file. If you type `size(words{1})` you will discover that this array has 278 elements – this is the total number of words in the Gettysburg address. The great advantage of cell arrays is that Matlab has many built-in functions that can operate on them, and this makes the rest of the program for counting word frequency distributions relatively easy.
- (3) For example, to count the number of unique words in the Gettysburg address, you need can use the built-in Matlab function `unique`. See the Matlab documentation.

The actual program has a few more complicated steps, such as eliminating all symbol strings that are not words, and then counting them, but hopefully you can now see how the computer can possibly count word frequency distributions.

Type `[results, freq] = wordcount('gettysburg.txt', 100)`; The number 100 tells the program that you want the top 100 words in the word frequency count.

Exercise 1

Use `wordcount.m` to compute the frequency distribution of words in the Gettysburg Address. Plot the distribution on a log-log plot, with the points as green dots.

Exercise 2

We have put two text files of news articles on the course website, both of which discuss the non-government shut down that happened a few years ago. The first article is from the Huffington Post, and the second is from Fox News. They have completely different slants, as you would expect. Measure the word frequency distribution of each article and plot these distributions on the same figure that you made for the Gettysburg Address, but plot the points in a different color or shape. Add axes labels and a legend.

Exercise 3

Now make a text file of some article or document of interest to you, that you can find on the web. Make your document into a .txt file. Measure the word frequency distribution of the document and plot it on your figure. You should see that that the word frequency distributions of all four documents have a surprising similarity. Compare the top few ranked words in the documents — are they the same or different? Look further down the list. When does the difference arise?

There is a famous law that governs the frequency distribution of words — called Zipf's law. See https://en.wikipedia.org/wiki/Zipf%27s_law. The law says that the frequency distribution of words in a document obeys

$$f_k = \frac{C}{k},$$

where f_k is the frequency of occurrence of the k th word and C is a constant.

Exercise 4

Do you agree with Zipf's law? Why or why not?

4. CALCULATING SHANNON'S ENTROPY

Finally, Shannon defined the entropy according to Equation (1) above.

Exercise 5

Write a function to compute the entropy of a set of word frequencies. The function should take in a list "freq" that is output by the wordcounts.m program, and compute the entropy in Equation (1). Make sure when you compute your entropy you use the base 2 logarithm.

Exercise 6

Use your function to measure the entropy of each of the texts. Is there more information content in either the Fox News article or the Huffington Post article?